Overview of Residual connection neural network

Subject: Residual connection neural network

$$a^{(l)} = \sigma!\,(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.

Basic block A basic block is the simplest building block studied in the original ResNet. This block consists of two sequential 3x3 convolutional layers and a residual connection. The input and output dimensions of both layers are equal.

2.

A residual neural network (also referred to as a residual network or ResNet) is a deep learning architecture in which the layers learn residual functions with reference to the layer inputs. It was developed in 2015 for image recognition, and won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) of that year. As a point of terminology, "residual connection" refers to the specific architectural motif of x ■ f (x) + x {\displaystyle x\mapsto f(x)+x} , where f {\displaystyle f} is an arbitrary neural network module. The motif had been used previously (see §History for details). However, the publication of ResNet made it widely popular for feedforward networks, appearing in neural networks that are seemingly unrelated to ResNet. The residual connection stabilizes the training and convergence of deep neural networks with hundreds of layers, and is a common motif in deep neural networks, such as transformer models (e.g., BERT, and GPT models such as ChatGPT), the AlphaGo Zero system, the AlphaStar system, and the AlphaFold system.

3.

Projection connection If the function F {\displaystyle F} is of type F : R n → R m {\displaystyle F:\mathbb {R} ^{n}\to \mathbb {R} ^{m}} where n ≠ m {\displaystyle n\neq m} , then F (x) + x {\displaystyle F(x)+x} is undefined. To handle this special case, a projection connection is used:

4.

∂ E ∂ x ■ = ∂ E ∂ x L ∂ x L ∂ x ■ = ∂ E ∂ x L (1 + ∂ ∂ x ■ ∑ i = ■ L − 1 F (x i)) = ∂ E ∂ x L + ∂ E ∂ x L ∂ ∂ x ■ ∑ i = ■ L − 1 F (x i) {\displaystyle {\begin{aligned}{\frac {\partial {\mathcal {E}}}{\partial x_{\ell }}}&={\frac {\partial {\mathcal {E}}}{\partial x_{L}}}{\frac {\partial x_{L}}{\partial x_{\ell }}}\\&={\frac {\partial {\mathcal {E}}}{\partial x_{L}}}\left(1+{\frac {\partial }{\partial x_{\ell }}}\sum _{i=\ell }^{L-1}F(x_{i})\right)\\&={\frac {\partial {\mathcal {E}}}{\partial x_{L}}}+{\frac {\partial {\mathcal {E}}}{\partial x_{L}}}{\frac {\partial }{\partial x_{\ell }}}\sum _{i=\ell }^{L-1}F(x_{i})\end{aligned}}}

5.

where P {\displaystyle P} is typically a linear projection, defined by P (x) = M x {\displaystyle P(x)=Mx} where M {\displaystyle M} is a m × n {\displaystyle m\times n} matrix. The matrix is trained via backpropagation, as is any other parameter of the model.

6.

Forward propagation If the output of the ■ {\displaystyle \ell } -th residual block is the input to the (■ + 1) {\displaystyle (\ell +1)} -th residual block (assuming no activation function between blocks), then the (■ + 1) {\displaystyle (\ell +1)} -th input is:

7.

Backward propagation The residual learning formulation provides the added benefit of mitigating the vanishing gradient problem to some extent. However, it is crucial to acknowledge that the vanishing gradient issue is not the root cause of the degradation problem, which is tackled through the use of normalization. To observe the effect of residual blocks on backpropagation, consider the partial derivative of a loss function E {\displaystyle {\mathcal {E}}} with respect to some residual block input x ■ {\displaystyle x_{\ell }} . Using the equation above from forward propagation for a later residual block L > ■ {\displaystyle L>\ell } :

8.

Bottleneck block A bottleneck block consists of three sequential convolutional layers and a residual connection. The first layer in this block is a 1×1 convolution for dimension reduction (e.g., to 1/2 of the input dimension); the second layer performs a 3×3 convolution; the last layer is another 1×1 convolution for dimension restoration. The models of ResNet-50, ResNet-101, and ResNet-152 are all based on bottleneck blocks.

9.

Residual connection In a multilayer neural network model, consider a (non-residual) subnetwork with a certain number of stacked layers (e.g., 2 or 3). Let H (x ; α) {\displaystyle H(x;\alpha)} denote the subnetwork. Suppose H ∗ {\displaystyle H^{*}} is the desired optimal output of this subnetwork. Residual learning simply adds x {\displaystyle x} directly to the output, such that the optimal learned output now becomes be H ∗ − x {\displaystyle H^{*}-x} , which is interpreted as a "residual" with respect to x {\displaystyle x} . The operation of "adding x {\displaystyle x} " is implemented via a "skip connection" that performs an identity mapping to connect the input of the subnetwork with its output. This connection is referred to as a "residual connection" in later work. Let F (x ; α) = H (x ; a) + x {\displaystyle F(x;\alpha)=H(x;a)+x} . The function F {\displaystyle F} is often represented by matrix multiplication interlaced with activation functions and normalization operations (e.g., batch normalization or layer normalization). As a whole, one of these subnetworks is referred to as a "residual block". A deep residual network is constructed by simply stacking these blocks. Long short-term memory (LSTM) has a memory mechanism that serves as a residual connection. In an LSTM without a forget gate, an input x t {\displaystyle x_{t}} is processed by a function F {\displaystyle F} and added to a memory cell c t {\displaystyle c_{t}} , resulting in c t + 1 = c t + F (x t) {\displaystyle c_{t+1}=c_{t}+F(x_{t})} . An LSTM with a forget gate essentially functions as a highway network. To stabilize the variance of the layers' inputs, it is recommended to replace the residual connections x + f (x) {\displaystyle x+f(x)} with x / L + f (x) {\displaystyle x/L+f(x)} , where L {\displaystyle L} is the total number of residual layers.

10.

where φ {\displaystyle \phi } can be any activation (e.g. ReLU) or normalization (e.g. LayerNorm) operation. This design reduces the number of non-identity mappings between residual blocks, and allows an identity mapping directly from the input to the output. This design was used to train models with 200 to over 1000 layers, and was found to consistently outperform variants where the residual path is not an identity function. The pre-activation ResNet with 200 layers took 3 weeks to train for ImageNet on 8 GPUs in 2016. Since GPT-2, transformer blocks have been mostly implemented as pre-activation blocks. This is often referred to as "pre-normalization" in the literature of transformer models.